

Nested Classes & Lambdas

Java Advanced Developer

StayAhead Training; Sep, 2025

Nested Classes

Nested Classes & Lambdas

- A class inside another
- A non-static nested class is an inner class
- Inner classes can access members of the enclosing class (even private ones)
- Static nested classes cannot access instance members of the encl. class
- Don't serialise nested classes (lambdas can be serialised)

Inner Classes

Nested Classes & Lambdas

```
class Outer {
```

```
    ...
```

```
    class Inner {
```

```
        ...
```

```
    }
```

```
}
```

Can have any
access modifier

```
Outer outer = new Outer();
```

```
Outer.Inner inner = outer.new Inner();
```


Static Nested Classes

Nested Classes & Lambdas

```
class Outer {  
    ...  
    static class Nested {  
        ...  
    }  
}
```

```
Outer.Nested nested = new Outer.Nested();
```

Shadowing

Nested Classes & Lambdas

- The hiding of a field by a local variable, or...
- The hiding of a class member by one with the same name in the nested class
- One cannot refer to a shadowed member by its name alone

Shadowing

Nested Classes & Lambdas

```
class Outer {  
    String greeting = "Hello";  
    class Inner {  
        String greeting = "Hi";  
        void greet(String greeting) {  
            System.out.println(greeting);           // param. value  
            System.out.println(this.greeting);       // "Hi"  
            System.out.println(Outer.this.greeting); // "Hello"  
        }  
    }  
}
```

Shadows by...

Shadows by...

Local Classes

Nested Classes & Lambdas

- A class defined in a block
- Can access members of the enclosing class
- Can access local variables provided they are `final` or effectively final*
- *Effectively final: a variable that is never reassigned after initialisation

Local Classes

Nested Classes & Lambdas

```
class Outer {  
    void doSomethingOuterish() {  
        ...  
        class Local {  
            void doSomethingLocalish() {...}  
        }  
        ...  
        new Local().doSomethingLocalish();  
    }  
}
```


Local Classes

Nested Classes & Lambdas

```
class Outer {  
    void doSomethingOuterish() {  
        int x = 1;  
        class Local {  
            void doSomethingLocalish() { System.out.println(x); }  
        }  
        x += 1;  
        new Local().doSomethingLocalish();  
    }  
}
```

Won't compile; x is not
final/effectively final

Local Classes

Nested Classes & Lambdas

```
class Outer {  
    void doSomethingOuterish() {  
        int x = 1;  
        class Local {  
            void doSomethingLocalish() { System.out.println(x); }  
        }  
        x += 1;  
        new Local().doSomethingLocalish();  
    }  
}
```

OK; x is
effectively final

Anonymous Classes

Nested Classes & Lambdas

- A local class with no name
- To create an instance of an interface
- Like the invocation of a constructor with a class definition appended
- Enclosing class member and local variable access as per local classes

Anonymous Classes

Nested Classes & Lambdas

```
interface Worker { void work(); }
```

```
class Outer {  
    void doSomethingOuterish() {  
        Worker worker = new Worker() {  
            void work() { System.out.println("Working..."); }  
        }  
        worker.work();  
    }  
}
```

Lambda Expressions

Nested Classes & Lambdas

- Another (more concise) way of coding anonymous classes, but...
- Only applicable when seeking to instantiate a functional interface*
- Functional interface: an interface with one abstract method only, e.g.:

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

Can have
default concrete and
static methods too

Lambda Expressions

Nested Classes & Lambdas

```
class Executor {  
    void executeAfter(int delayInMs, Runnable runnable) {  
        ...  
    }  
}  
...  
var executor = new Executor();  
executor.execute(1000, new Runnable() {  
    void run() { System.out.println("Hello world"); }  
});
```


Lambda Expressions

Nested Classes & Lambdas

- Re. the code on the previous slide, when we call `executeAfter`:
 - the compiler knows that a `Runnable` is expected
 - the compiler knows `Runnable` is a functional interface
 - the compiler knows the signature of the one abstract method
 - the compiler does not know how the one method should be implemented

Lambda Expressions: Syntax

Nested Classes & Lambdas

1. Parameter list
2. Meagre arrow ->
3. Code block (method body)

Lambda Expressions: Syntax

Nested Classes & Lambdas

```
class Executor {  
    void executeAfter(int delayInMs, Runnable runnable) {  
        ...  
    }  
}  
...  
var executor = new Executor();  
executor.execute(1000, new Runnable() {  
    void run() -> { System.out.println("Hello world"); }  
});
```

Lambda Expressions: Syntax

Nested Classes & Lambdas

```
class Executor {  
    void executeAfter(int delayInMs, Runnable runnable) {  
        ...  
    }  
}  
...  
var executor = new Executor();  
executor.execute(1000,  
    () -> { System.out.println("Hello world"); }  
);
```

Lambda Expressions: Syntax

Nested Classes & Lambdas

```
class CourseStore {  
    List<String> searchCourses(Predicate predicate) {  
        ...  
    }  
}  
...  
var store = new CourseStore();  
var courses = store.searchCourses(new Predicate() {  
    boolean test(String title) {  
        return title.contains("Java");  
    }  
});
```


Lambda Expressions: Syntax

Nested Classes & Lambdas

```
class CourseStore {  
    List<String> searchCourses(Predicate predicate) {  
        ...  
    }  
}  
...  
var store = new CourseStore();  
var courses = store.searchCourses(new Predicate() {  
    boolean test(String title) -> {  
        return title.contains("Java");  
    }  
});
```

Lambda Expressions: Syntax

Nested Classes & Lambdas

```
class CourseStore {  
    List<String> searchCourses(Predicate predicate) {  
        ...  
    }  
}  
...  
var store = new CourseStore();  
var courses = store.searchCourses(  
    (String title) -> {  
        return title.contains("Java");  
    }  
);
```

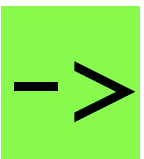
Lambda Expressions: Rules

Nested Classes & Lambdas

- Parameter data types need not be specified
- Round brackets may be omitted if one param. only (but not zero params.)
- Braces may be omitted if one line of code only in method body
- `return` keyword **must** be omitted if braces are omitted

Lambda Expressions: Rules

Nested Classes & Lambdas

```
class CourseStore {  
    List<String> searchCourses(Predicate predicate) {  
        ...  
    }  
}  
...  
var store = new CourseStore();  
var courses = store.searchCourses(  
    (String title) {  {  
        return title.contains("Java");  
    }  
);
```

Lambda Expressions: Rules

Nested Classes & Lambdas

```
class CourseStore {  
    List<String> searchCourses(Predicate predicate) {  
        ...  
    }  
}  
...  
var store = new CourseStore();  
var courses = store.searchCourses(  
    title -> title.contains("Java");  
);
```


Lambda Expressions: Anonymous Method?

Nested Classes & Lambdas

- Devs will often say, e.g., a lambda is an anonymous method, where in fact...
- ...it is an instance of an anonymous implementation of a functional interface

```
Runnable runnable = () -> System.out.println("Hello");  
  
runnable.run();
```

- Note that one can establish the signature of the method from the lambda

Lambda Expressions: Local Var. Capture

Nested Classes & Lambdas

- Note that a lambda is part local class
- A lambda can access local variables provided they are final/effectively final

```
void doSomething() {  
    int x = 1;  
    Runnable runnable = () -> System.out.println(x);  
    runnable.run();  
    x += 1;  
}
```

Won't compile; x is not
final/effectively final

java.util.function.Supplier

Nested Classes & Lambdas

```
public interface Supplier<T> {  
    T get();  
}
```

...

```
Supplier<String> supplier = () -> "Hello";
```


java.util.function.Consumer

Nested Classes & Lambdas

```
public interface Consumer<T> {  
    void accept(T t);  
}
```

...

```
Consumer<Integer> consumer = n -> System.out.println(n);
```

java.util.function.Predicate

Nested Classes & Lambdas

```
public interface Predicate<T> {  
    boolean test(T t);  
}
```

...

```
Predicate<Integer> predicate = n -> n % 2 == 1;
```

java.util.function.Function

Nested Classes & Lambdas

```
public interface Function<T, R> {  
    R apply(T t);  
}
```

...

```
Function<String, Integer> function = str -> str.length();
```


java.util.function.Function

Nested Classes & Lambdas

- Function lambdas may result in unnecessary boxing and unboxing, e.g.:

```
Function<Double, Integer> func = d -> d.intValue();
```

```
double n1 = 7.5;
```

```
int n2 = func.apply(n2);
```

Boxing and/or unboxing occurs
THREE times here!

java.util.function.Function

Nested Classes & Lambdas

- Many Function variations to avoid boxing and unboxing, e.g.:

```
DoubleToIntFunction func = d -> (int) d;
```

```
double n1 = 7.5;
```

```
int n2 = func.applyAsInt(n2);
```

No boxing/unboxing

java.util.function (Others)

Nested Classes & Lambdas

- A UnaryOperator is a Function where both types are the same
- BiConsumer, BiPredicate, & BiFunction each accept two args., e.g.:

```
public interface BiPredicate<T, U> {  
    boolean test(T t, U, u);  
}
```

- A BinaryOperator is a BiFunction where all three types are the same

Method References

Nested Classes & Lambdas

- An instance of anonymous implementation of a functional interface
- More concise than a lambda expression when calling existing methods, e.g.:

```
Predicate<String> predicate = string -> string.isBlank();
```

```
Predicate<String> predicate = String::isBlank;
```

- Four variations - all include the double colon syntax

Method References: Static

Nested Classes & Lambdas

- Syntax: `Class::staticMethod`

// as a lambda

```
Supplier<Double> supplier = () -> Math.random();
```

// as a method ref.

```
Supplier<Double> supplier = Math::random;
```

// no unboxing

```
DoubleSupplier supplier = Math::random;
```


Method References: Static

Nested Classes & Lambdas

- Syntax: `Class::staticMethod`

`// as a lambda`

```
Function<Double, Double> function = n -> Math.ceil(n);
```

`// as a method ref.`

```
Function<Double, Double> function = Math::ceil;
```

`// no boxing/unboxing`

```
DoubleUnaryOperator function = Math::ceil;
```

Method References: Unbound

Nested Classes & Lambdas

- Syntax: `Class::instanceMethod`

// as a lambda

```
Predicate<String> predicate = str -> str.isBlank();
```

// as a method ref.

```
Predicate<String> predicate = String::isBlank;
```

- Note the (first) interface method argument is assumed to be the object on which the existing method (`isBlank`) is called

Method References: Unbound

Nested Classes & Lambdas

- Syntax: `Class::instanceMethod`

// as a lambda

```
BiFunction<String, String, Integer> biFunction =  
    (str1, str2) -> str1.indexOf(str2);
```

// as a method ref.

```
BiFunction<String, String, Integer> biFunction =  
    String::indexOf;
```


Method References: Bound

Nested Classes & Lambdas

- Syntax: `instance::instanceMethod`

```
// as a lambda (today is a LocalDate)
```

```
Supplier<Integer> supplier = () -> today.getYear();
```

```
// as a method ref.
```

```
Supplier<Integer> supplier = today::getYear;
```

```
// no unboxing
```

```
IntSupplier supplier = today::getYear;
```

Method References: Bound

Nested Classes & Lambdas

- Syntax: `instance::instanceMethod`

```
// as a lambda (str is a String)
BinaryOperator<String> biFunction =
    (old, new) -> str.replace(old, new);
```

```
// as a method ref.
BinaryOperator<String> biFunction = str::replace;
```


Method References: Constructor

Nested Classes & Lambdas

- Syntax: `Class::new`

// as a lambda

```
Supplier<List<String>> supplier = () -> new ArrayList<>();
```

// as a method ref.

```
Supplier<List<String>> supplier = ArrayList::new;
```

Combining Lambdas

Nested Classes & Lambdas

- Predicate has default methods for chaining n predicates, e.g.:

```
Predicate<Integer> gt10 = n -> n > 10;
```

```
Predicate<Integer> lt20 = n -> n < 20;
```

```
Predicate<Integer> between10And20 = gt10.and(lt20);
```

Combining Lambdas

Nested Classes & Lambdas

- Predicate has default methods for chaining n predicates, e.g.:

```
Predicate<String> isError = s -> s.contains("Error");  
Predicate<String> isWarning = s -> s.contains("Warning");
```

```
Predicate<String> isNotOk = isError.or(isWarning);
```

```
Predicate<String> isOk = isNotOk.negate();
```


Combining Lambdas

Nested Classes & Lambdas

- Consumer has a default method for chaining n consumers, e.g.:

```
var names = new ArrayList<String>();
```

```
Consumer<String> addToNames = names::add;
```

```
Consumer<String> print = System.out::println;
```

```
Consumer<String> consumer = addToName.andThen(print);
```

Combining Lambdas

Nested Classes & Lambdas

- Function (& derivatives) has default methods for chaining n functions, e.g.:

```
IntUnaryOperator decr = n -> n - 1;
```

```
IntUnaryOperator sq = n -> n * n;
```

```
IntUnaryOperator decrThenSq = decr.andThen(sq);
```

```
IntUnaryOperator sqThenDecr = decr.compose(sq);
```

Apply the given operator first